

Final Project

Log-Driven Sonification System

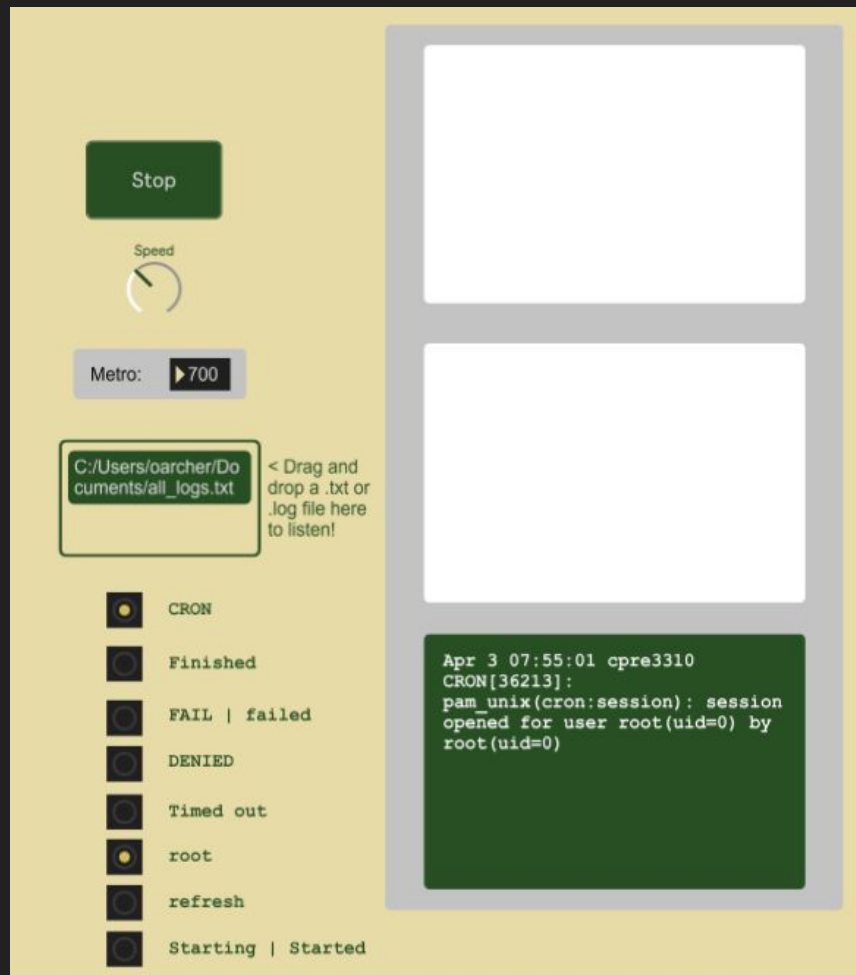
What is it?

This project turns system log data into a generative soundscape, where normal activity creates ambient textures and errors create disruptions.

Motivation

Blend backgrounds in cybersecurity and sound design. The proposed use case would be to integrate this system into CDC environments to make system activity audible in real time!

Overall, this could introduce another method for cybersecurity monitoring by reducing reliance on strictly visual tools, while challenging how unconventional datastreams can drive musical systems.



Sonification

Data Sonification:

- Sonification = translating data into sound for perception and analysis
- Used to:
 - Detect patterns
 - Monitor systems
 - Represent complex data streams

Event-Based:

- Discrete triggers (log entries)
- Finished == sparkle sound, CRON == drum-like pulses

Continuous:

- Ongoing sound textures
- Reflect system “state” over time

Brian Eno (Generative Music)

His Key Ideas:

- Music is a living system
- Built from constraints and processes
- Evolves without direct control

He Inspired:

- My patch is not a fixed composition
 - Evolves based on incoming data
 - Multiple layers interact with each other over time

“My kind of composing is more like the work of a gardener. The gardener takes his seeds and scatters them, knowing what he is planting but not quite what will grow where and when – and he won’t necessarily be able to reproduce it again afterwards either.”

Eric Tamm (Analysis of Eno)

His Key Ideas:

- Systems thinking
- “Vertical color of sound” by layering textures

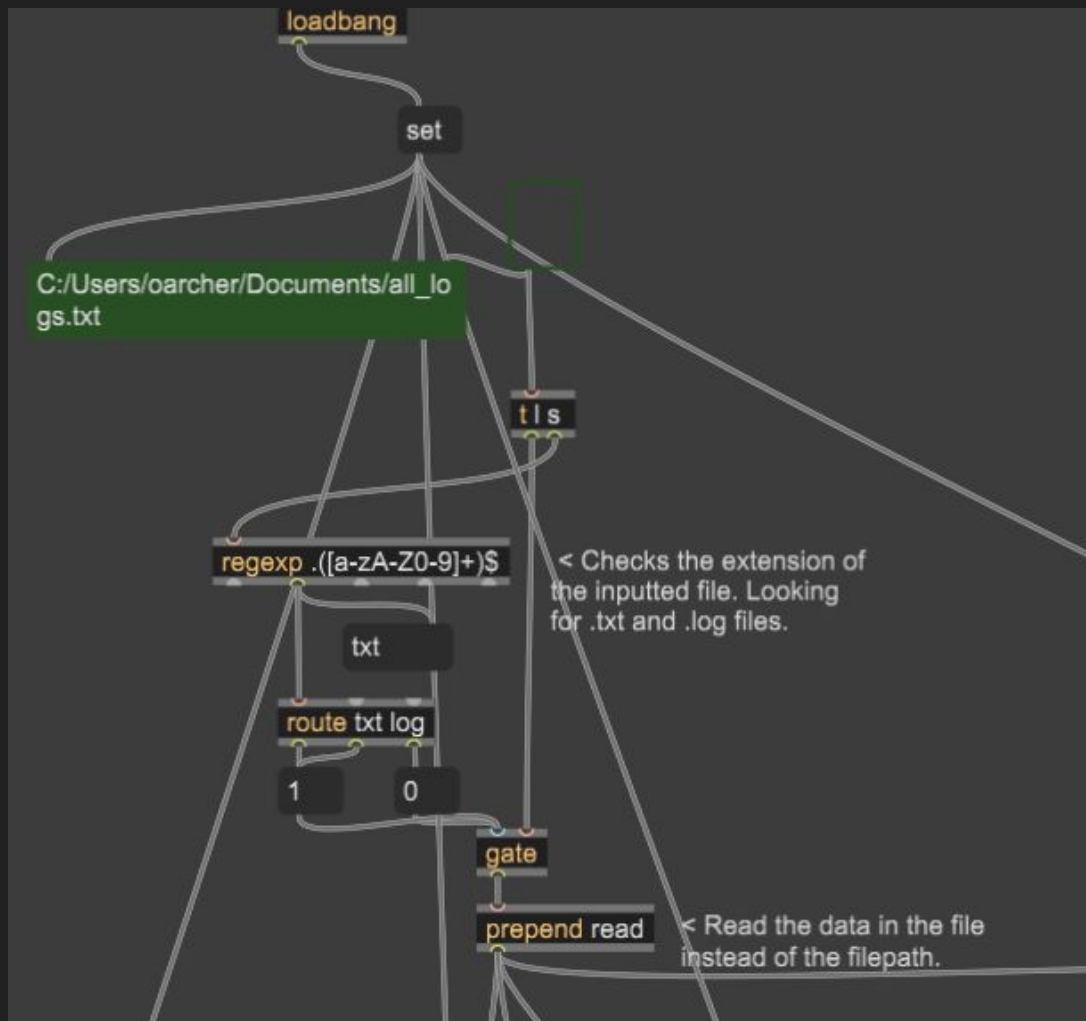
He Inspired:

- Independent sound layers (through subpatches)
 - Each layer contributes to the overall texture of the environment

Input & Data Handling

This section of the main patch handles the file input.

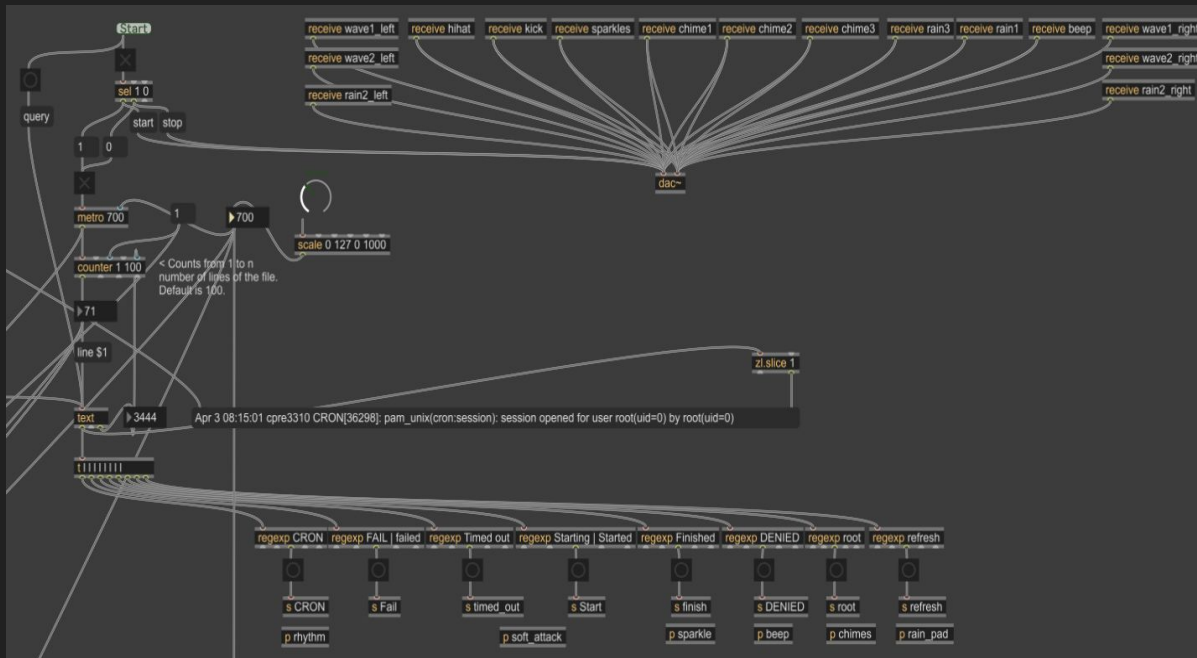
- Automatically loads nothing as the default log file.
- Once a file is dropped into the [dropfile] object, the green message box contains the file path.
- The [regexp] object checks the file extension for .txt or .log to ensure only valid log files enter the system.
 - The gate only allows .txt and .log files to be read.
- The [prepend read] object converts the file path into the data inside for the [text] objects.



The Parsing System

This section of the main patch converts the unstructured log messages into categorized auditory triggers using pattern recognition.

- Start/Stop trigger connected to [dac~] outputs and enable parsing.
- The [text] object holds all of the lines from the loaded file.
- The [counter] and [metro] objects iterate through the lines on a user inputted clock.
- The [line \$1] message only triggers one line to be outputted at a time.
- Multiple [regex] objects are used to determine if the specified keywords are found in a given line
 - These are linked to [send] objects for subpatch implementation

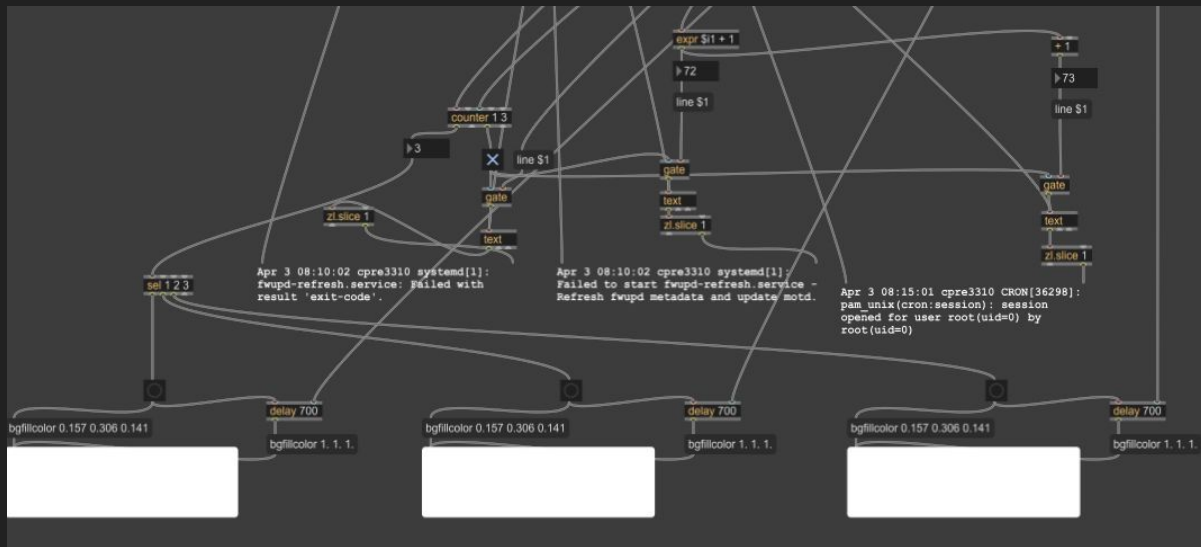


Visualization of Log Lines

This section of the main patch helps with the visualization of the log lines.

- 3 lines from the text file are shown at a time.
- The messages are printed in white and are hidden until the line is “read” by the system, when the panel behind it turns green.

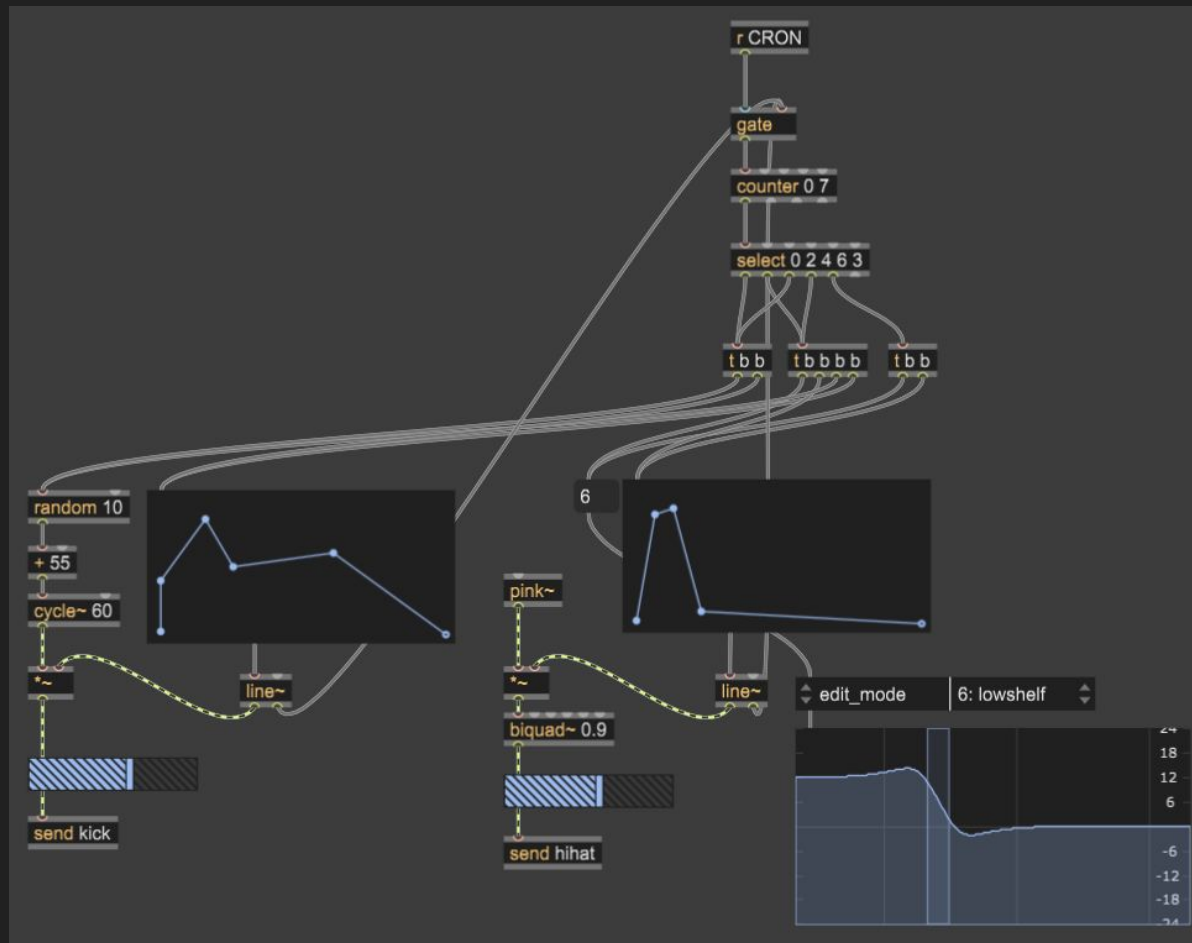
Originally, I wanted to implement a “sliding window” for the log file lines. But it is not a simple task to implement a double-ended queue like structure in Max9 programming.



Rhythm Subpatch

This subpatch converts CRON events into rhythmic patterns of kick and hi-hats.

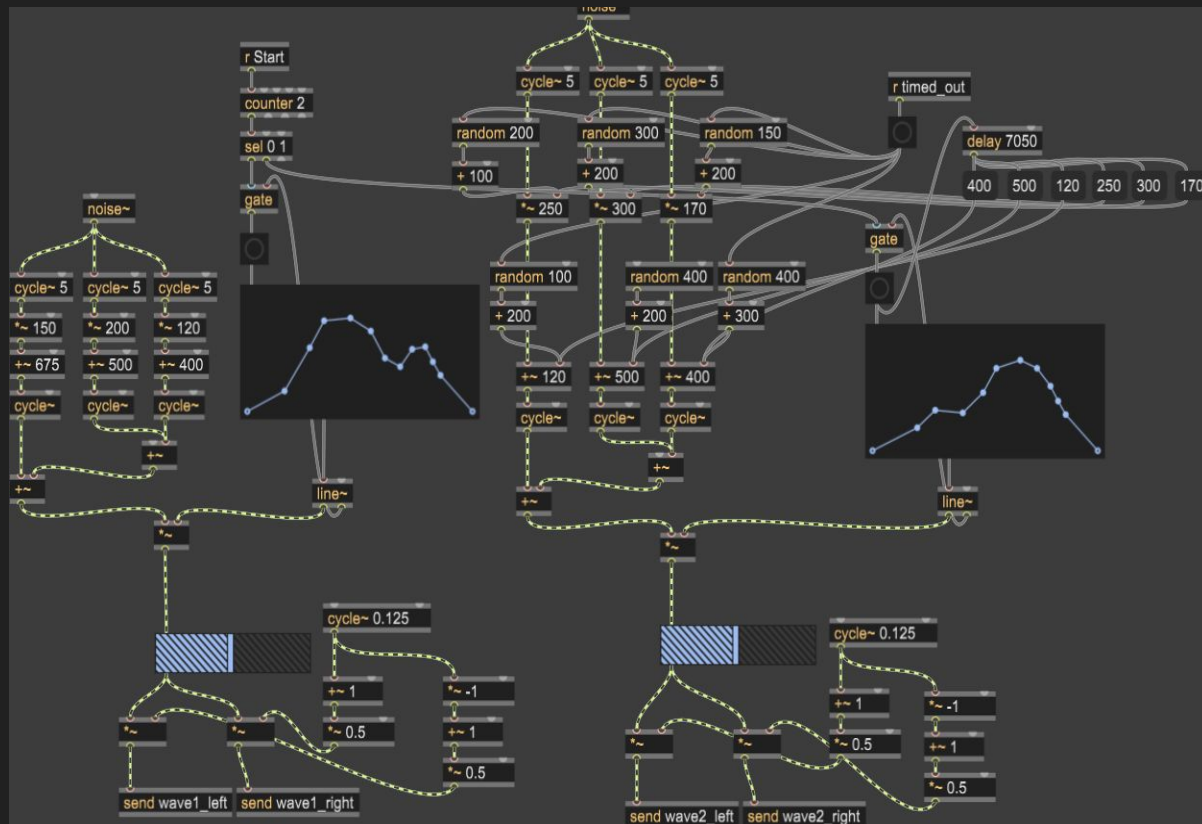
- The [receive] object takes the CRON event and enables the rhythm generation.
- Uses the [counter] object to step through an 8-step cycle.
- Kick drum generation is on the left.
 - Slight variation of tone using the [random] object.
 - Envelope shapes kick.
- Hi-hat generation is on the right.
 - Uses [pink~] object as the noise source.
 - [biquad~] filtering and tone shaping.
 - Short percussive hits from envelope.



Soft Attack Subpatch

This subpatch converts Start events into rhythmic patterns of wind-like waves. If the a Timed out event is triggered, one of the waves will have randomized inputs to create more off-putting sounds.

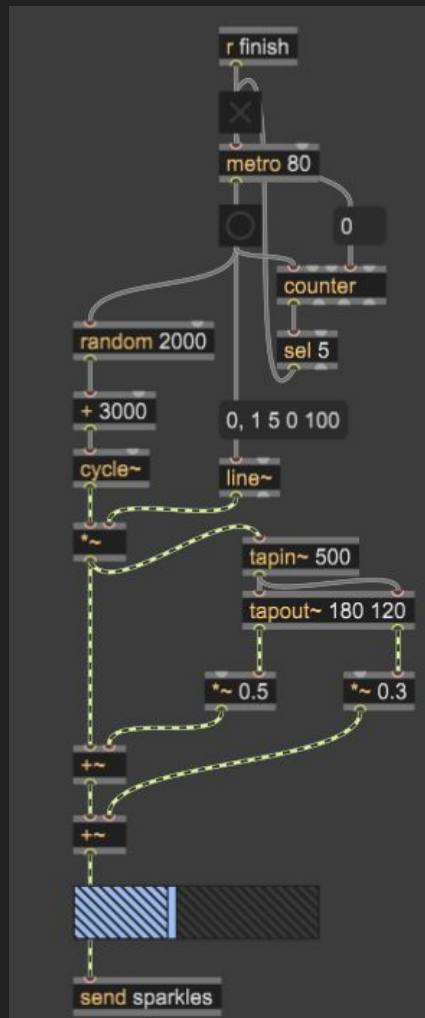
- The Start event cycles through two different wave sounds.
 - The Timed out events only alter the right wave's sound.
- Both waves shift in a wave-like form between the two channels.



Sparkle Subpatch

This subpatch converts Finished events into a short decaying sparkle texture.

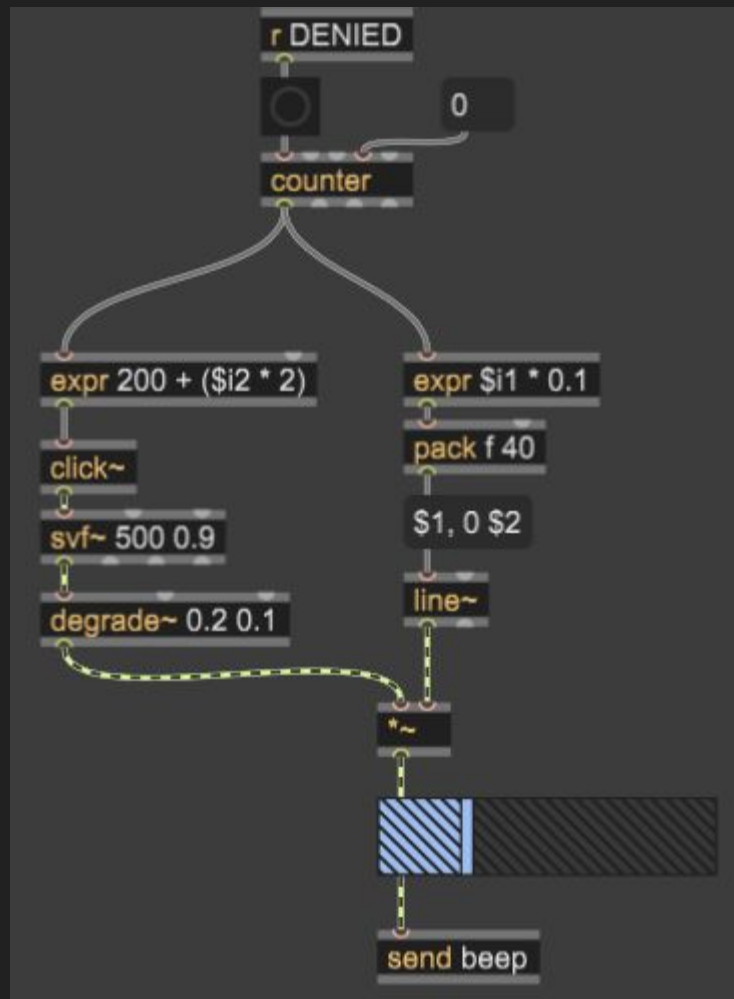
- The [toggle] object and the [metro] object creates a rapid repeated trigger.
- The [random], [+ 3000], and [cycle~] objects create high-frequency tones with a randomized pitch to create the bright sparkle texture.
- The message attached to the [line~] object defines a fast attack (5 ms) with a decay of 100 ms.
 - Granular-like behavior.
- The [tapin~] and [tapout~] objects creates the layered echoes.



Beep Subpatch

This subpatch converts DENIED events into a harsh beep to represent denied or unauthorized system events.

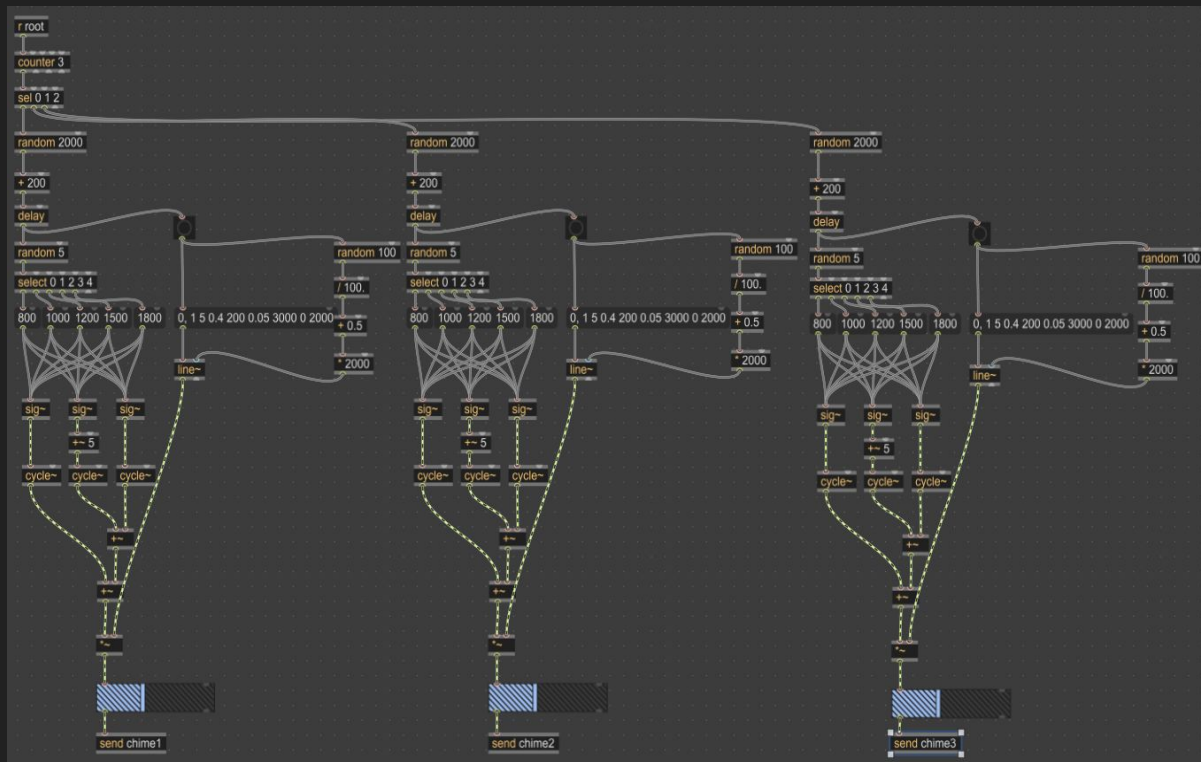
- Each time the DENIED event occurs, the beep increases in amplitude.
 - $[\text{expr } \$i1 * 0.1]$ scales the amplitude based on event count.
- Short decay envelope for sharp alerts.



Chimes Subpatch

This subpatch generates a wind chime-like ambient texture to represent ongoing background system activity based on the root event.

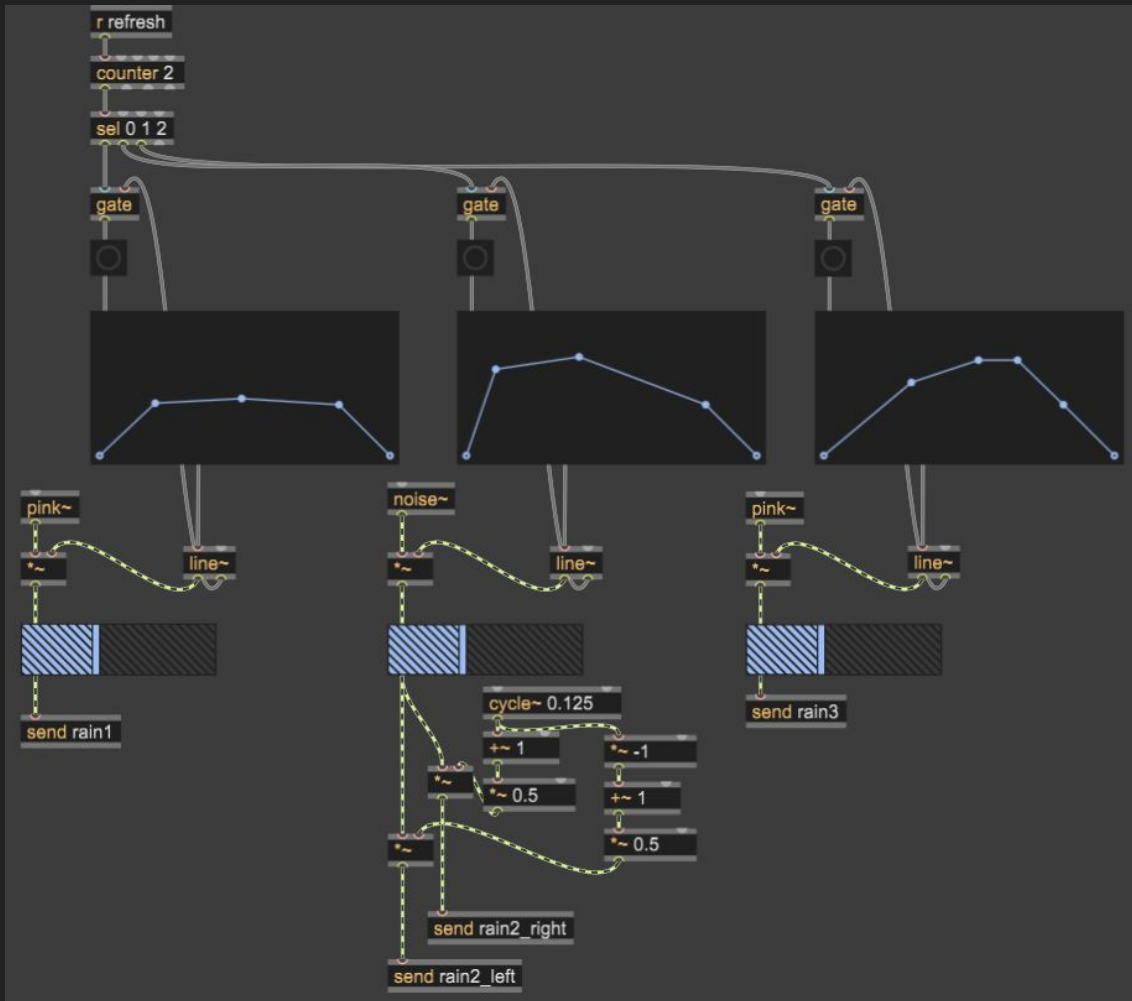
- Three parallel voices to create layered texture and avoid repeating the same envelope, cutting it off.
- The [random] and [delay] objects generate slight timing offsets, to prevent predictable repetition.
- The [random] and [select] objects select pitches from a defined set to create consistent, but various sounds.
- Each chime is built from multiple sine waves, to create a bell-like tone.



Rain Pad Subpatch

This subpatch generates a continuous rain-like texture to signify a system or process refresh event.

- Three parallel voices with different functions.
 - Creates smooth swells instead of harsh hits.
- The [noise~] and [pink~] objects are combined to create more varied rain textures.
- The middle section slowly pans sound left to right to further the interest and movement of the environment.

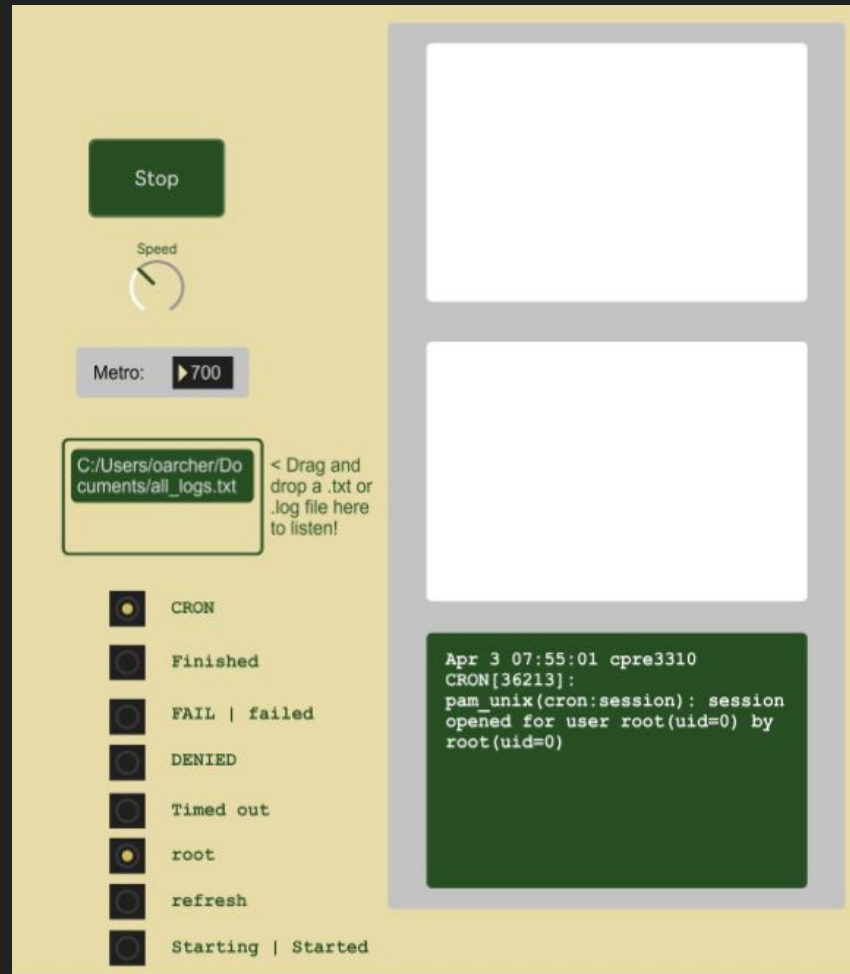


Start/Stop Button>

Dial to control >
speed of line
parsing

Place to drop .txt >
or .log file

Visual indicators >
of found parsed
terms



< Visual indicator of
which line is being
read

* Note that this is not a completed design
and there are subpatches missing to
complete the sonic environment.

Sources

Sonification:

[https://mct-master.github.io/sonification/2019/04/05/Thomas-Hermann.html#:~:text=Hermann\(2008\)%20has%20given%20a,is%20systematic%2C%20\(3\)%20the](https://mct-master.github.io/sonification/2019/04/05/Thomas-Hermann.html#:~:text=Hermann(2008)%20has%20given%20a,is%20systematic%2C%20(3)%20the)

Brian Eno: <https://www.soundoflife.com/blogs/people/brian-eno-generative-music>

Iannis Xenakis: <https://www.iannis-xenakis.org/en/stochastic-music/>

Eric Tamm:

https://monoskop.org/images/f/f1/Tamm_Eric_Brian_Eno_His_Music_and_the_Vertical_Color_of_Sound.pdf

Miranda & Wanderley:

<https://archive.org/details/newdigitalmusica0021mira/page/n7/mode/2up>